

Visualization

Data Analysis & Information Extraction

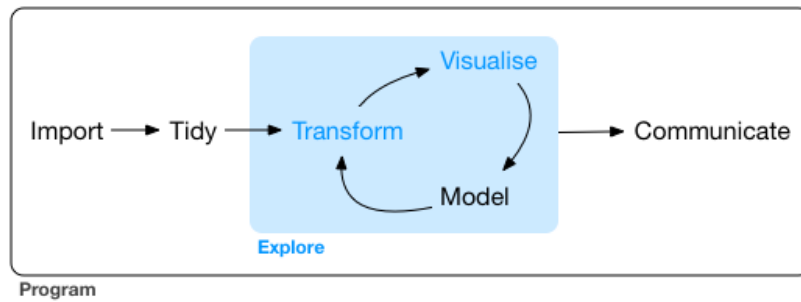
Table of contents

Exploratory Data Analysis (EDA)	2
Visualization: <i>ggplot2</i>	2
Plots with ggplot2	4
The Canvas	4
Geometries	5
Aesthetics	6
Understanding how to use mappings	7
Adapting the plot and Customizing	8
Titles and Labels	9
Aesthetics	10
Aesthetics for defining axis	11
Coloring the figures: color	12
Inside and outside the aesthetics	12
Parameters color vs fill	13
Shaping the figures: shape	15
Changing the size using size	17
Adding transparency using alpha	17
To Sum Up	19
Facets	20
Faceting using Grids: <i>facet_grid</i>	21
Faceting using Wraps: <i>facet_wrap</i>	23
Freeing the scales	24
To Sum Up	26
Other Features	26
Coordinates	26
Maps	27
Polar Plots	29

Exploratory Data Analysis (EDA)

An Exploratory Data Analysis or EDA consists on observe the data and analyze it to generate hypothesis and test them.

Objective: Generate questions to analyze them more deeply.



EDAs include visualization and data manipulation. This process, combined with the curiosity and skepticism, try to find interesting questions and answers about the data.

At this part of the course we will focus on create and study visual representations with the data. Using **ggplot2** a package inside the *tidyverse* ecosystem to create visualizations.

Visualization: *ggplot2*

The visualization package of *tidyverse*. Grammar of Graphics comes from the work by Leland Wilkinson. It will help us describe the different components of the plot in a sequential way, with defining the different layers of the final visualization.



For using **ggplot2** we need to load the library first, as it is part of *tidyverse* we will load the full package

Visualization: ggplot2

```
library(tidyverse) # Do library(ggplot2) for the package only
```

Plots with ggplot2

We are going to introduce the `ggplot2` package, that will be our main tool for designing and create plots to visualize information. For starting using it remember that we need to load the `tidyverse` library as this package is part of the `tidyverse` environment.

```
library(tidyverse)
```

If you are working with an older version of R maybe you need to load the library directly using `library(ggplot2)`.

We are going to start creating our first plot. We have the following data set called `gapminder`, it contains information regarding countries: GDP, Life Expectancy and Population for several years.

Here a review of the dimensions and variables the data frame has:

```
glimpse(gapminder)
```

```
Rows: 1,704
```

```
Columns: 6
```

```
$ country <fct> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", ~  
$ continent <fct> Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, Asia, ~  
$ year <int> 1952, 1957, 1962, 1967, 1972, 1977, 1982, 1987, 1992, 1997, ~  
$ lifeExp <dbl> 28.801, 30.332, 31.997, 34.020, 36.088, 38.438, 39.854, 40.8~  
$ pop <int> 8425333, 9240934, 10267083, 11537966, 13079460, 14880372, 12~  
$ gdpPercap <dbl> 779.4453, 820.8530, 853.1007, 836.1971, 739.9811, 786.1134, ~
```

We wanted to see the behavior between `lifeExp` and `gdpPercap` on year 1967 to detect if we can see any relation between both variables. As they are both numeric continuous variables an scatter-plot (dot plot) will be enough.

Let's prepare the data first filtering the information we want to visualize:

```
gapminder67 <- gapminder[gapminder$year == 1967, ]
```

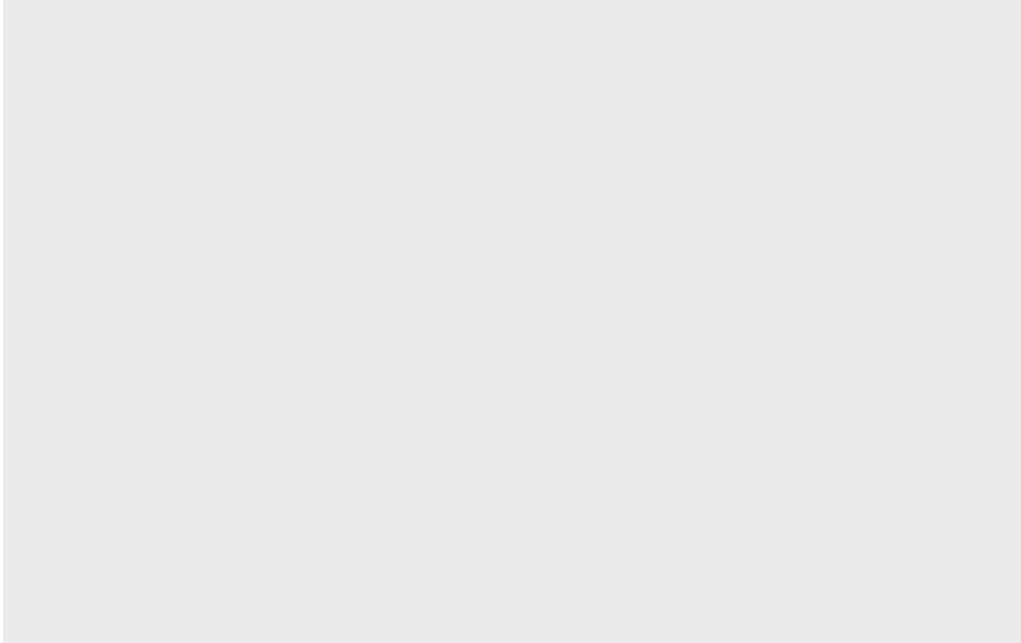
The Canvas

For creating a plot with `ggplot2` we need to define the canvas, the plot object where we will draw the plot. For doing this we will use function `ggplot()`.

```
ggplot(data = <data_to_use>, mapping = aes(x = <col1>, y = <col2>, ...))
```

We need to provide the ggplot function the data to use:

```
ggplot(gapminder)
```



Geometries

We have the initial canvas, but we need to define the object to draw. In ggplot we can draw different kinds of plots and combine them, all these objects are called **geometries**...

In this case we want to create an scatter plot, so we will use `geom_poin()` object. We have different options more, for example:

- scatter plot: `geom_point()`
- line plot: `geom_line()`
- histograms: `geom_hist()`
- ...

In this case we are adding a `geom_point` to the plot canvas, for doing that we need to use a `+` syntax and add it to the initial `ggplot`.

```
ggplot(gapminder) +  
  geom_point()
```

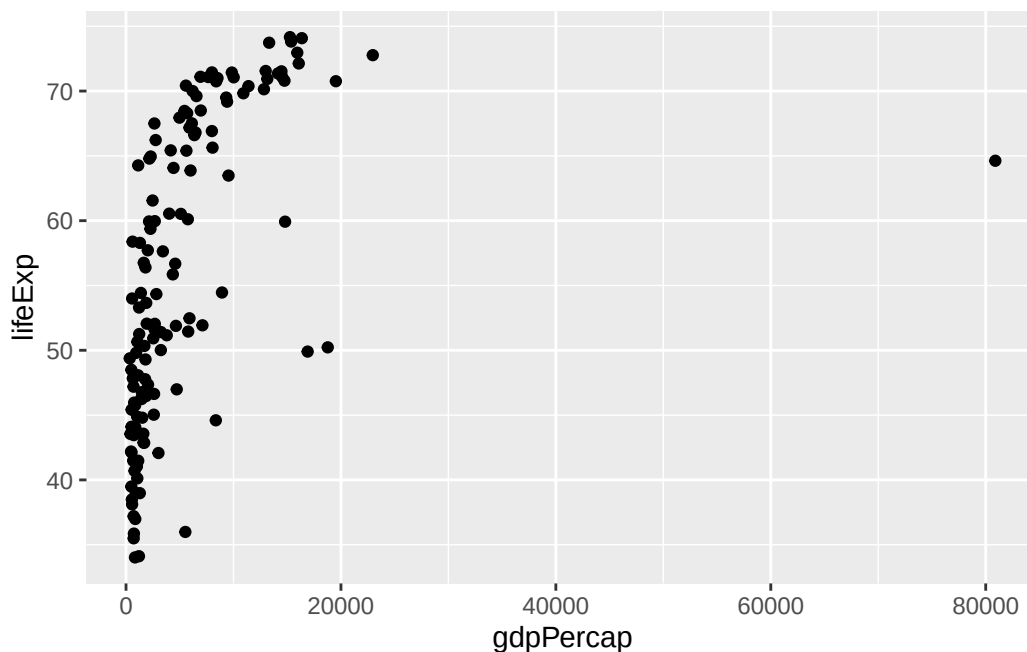
But there is something still missing... Which data we want to draw?!

Aesthetics

As we said we wanted to see the relation between lifeExp and gdpPercap, so our axis in the plot will be y as lifeExp and x as gdpPercap. How we introduce this information?

All geometries formulas have an argument called mapping where we need to introduce the aesthetics or aes(), this argument allow us to access the data inside the data frame. We will introduce the information as follows:

```
ggplot(data = gapminder67) +  
  geom_point(mapping = aes(y = lifeExp, x = gdpPercap))
```



Now we got it! Aesthetics will help us incorporate more information to our plot, helping introducing more variables with different options.

Here are some of the ones we can use...

- shape
- color / fill (first one borders, second full figure)

- size
- alpha (transparency)

As you can see for creating plots with ggplot we introduce each element using +, it works as a layer system where we introduce different elements one by one until having the desired plot.

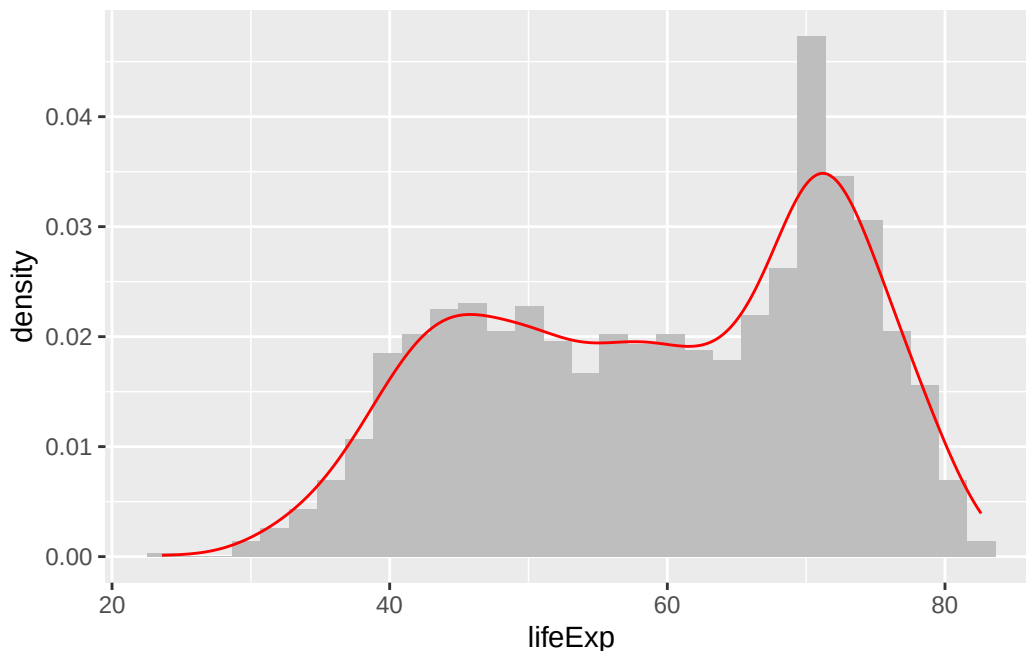
Understanding how to use mappings

For introducing the variables we need to use the mappings parameter and if the variable we need to use is from the data using the `aes()` function. In the example we are introducing the aesthetics into the geometries, but we can introduce them in the same ggplot, the difference depends on how it will interact with the layers of the plot.

We want to see how the possible values of `lifeExp` are distributed, we can see this with an histogram and a density plot. These geometries are `geom_histogram` and `geom_density`

```
ggplot(data = gapminder) +  
  geom_histogram(mapping = aes(x = lifeExp, y = after_stat(density)), fill = "grey") +  
  geom_density(mapping = aes(x = lifeExp), color="red")
```

``stat_bin()`` using ``bins = 30``. Pick better value ``binwidth``.

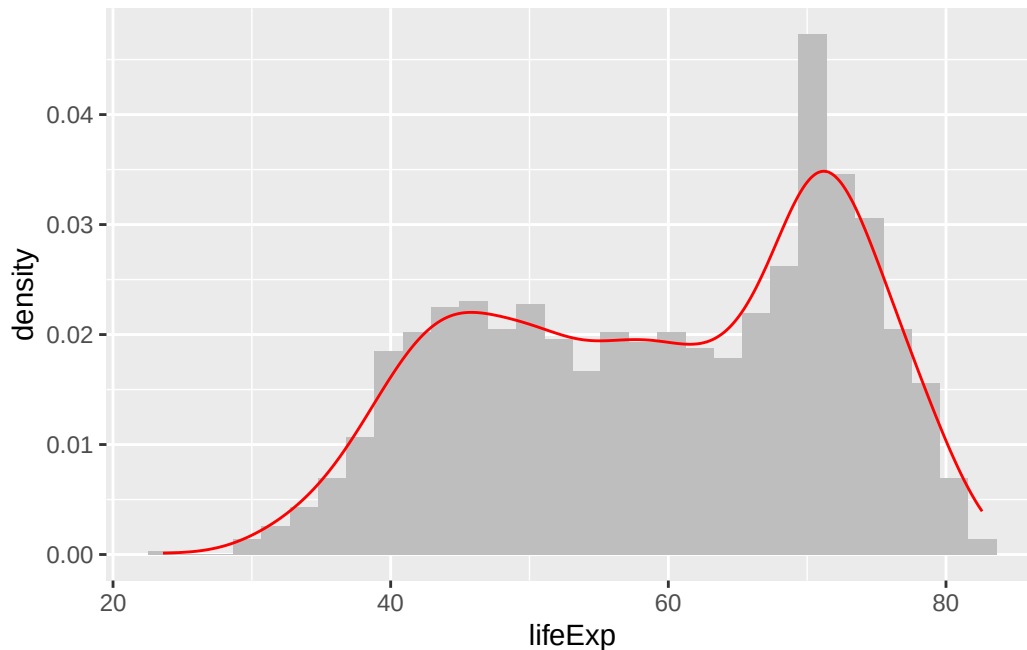


We have defined the axis in both elements added to the plot. But as both geometries share the same x axis we can define the aesthetics on the ggplot instead.

Adapting the plot and Customizing

```
ggplot(data = gapminder, mapping = aes(x = lifeExp)) +  
  geom_histogram(mapping = aes(y = after_stat(density)), fill = "grey") +  
  geom_density(color="red")
```

``stat_bin()`` using ``bins = 30``. Pick better value ``binwidth``.



When we define an aesthetics on the `ggplot()` object it will be shared with all the geometries we add to the figure.

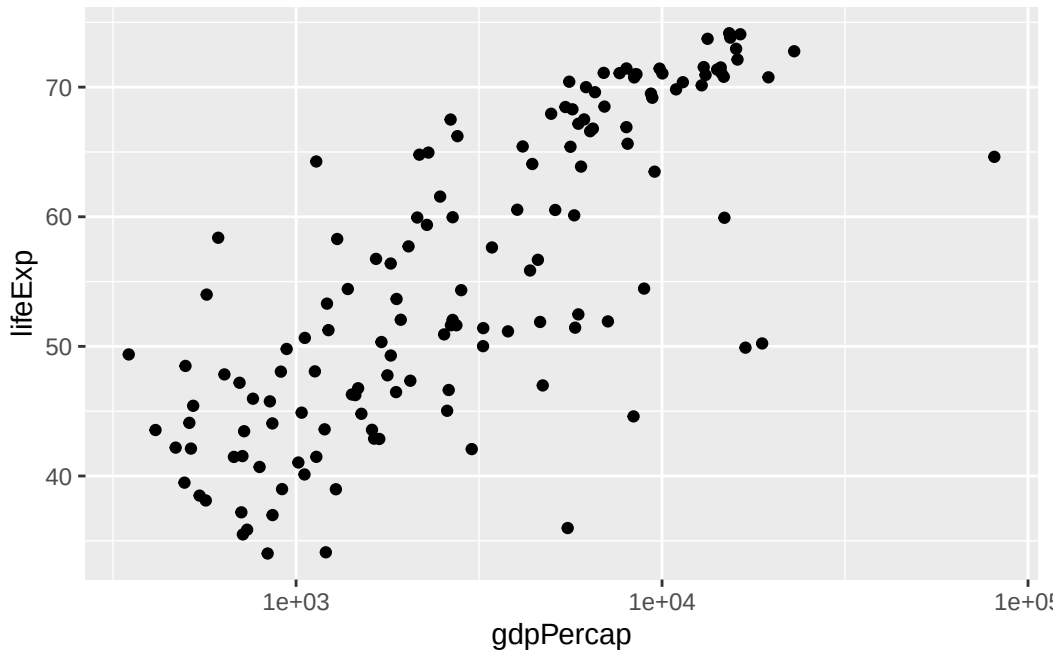
Adapting the plot and Customizing

Returning to our original plot about `lifeExp` and `gdpPercap` we are going to improve a bit the overall plot appearance and display.

The dots are located very near each other showing a more dense visualization and not allowing a clear view of the data.

For solving that we can include a change in the scale in `x` using the *log* base, this technique is used frequently for improving how the data is displayed. In `ggplot2` we can do it adding a new feature function called `scale_x_log10()`


```
ggplot(data = gapminder67) +
  geom_point(mapping = aes(y = lifeExp, x = gdpPerCap)) +
  scale_x_log10()
```



As you can see we include all these new features using the + syntax, as this needs to be included in the same canvas we are using for creating the plot.

With the change of scale, now can see more clearly the relation between both variables.

Both variables show a positive relation, a higher GDP per capital is related with a higher Life Expectancy

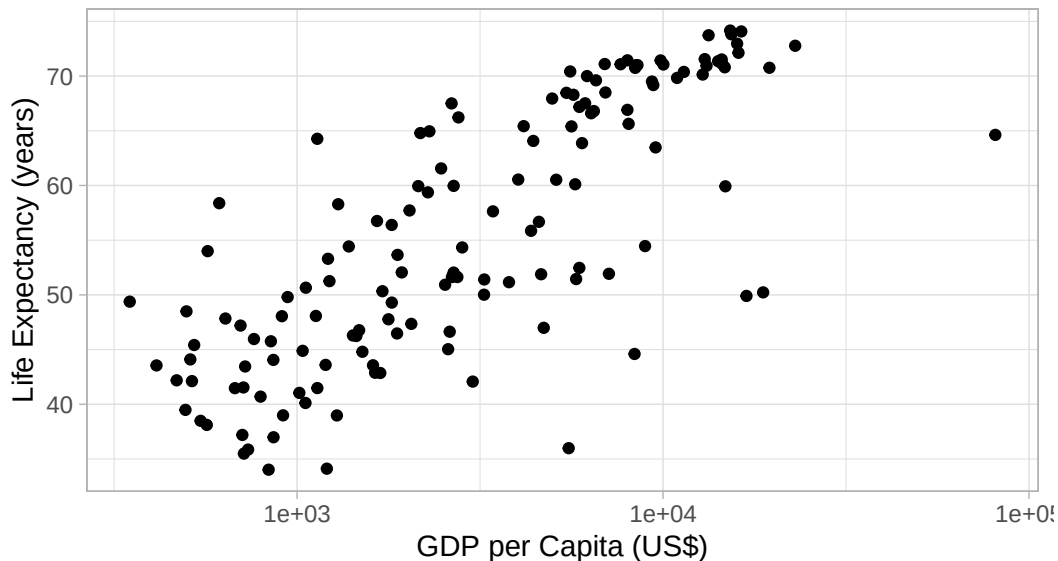
Titles and Labels

Finally, we can include some customizing of the titles and labels. In ggplot this can be done using labs() function.

```
ggplot(data = gapminder67) +
  geom_point(mapping = aes(y = lifeExp, x = gdpPerCap)) +
  scale_x_log10() +
  labs(
    title = "Relation between Life Expectancy and GDP per capital",
    subtitle = "Year 1967",
```

```
y = "Life Expectancy (years)",
x = "GDP per Capita (US$)"
) +
theme_light()
```

Relation between Life Expectancy and GDP per capital
Year 1967



This way we can improve and generate a final visualization to include in our reports and analysis. In the last code we have also included a *theme*, ggplot has predefined themes than change the plot overall appearance, as you can see all these new features on the plot are included using +.

Aesthetics

We have already seen in the last chapter how aesthetics work to define the axis of our plots, but in the aesthetics we can define and work with more parameters to create our visualizations.

Aesthetics are used for accessing data variables to be used in the plot for drawing the data.

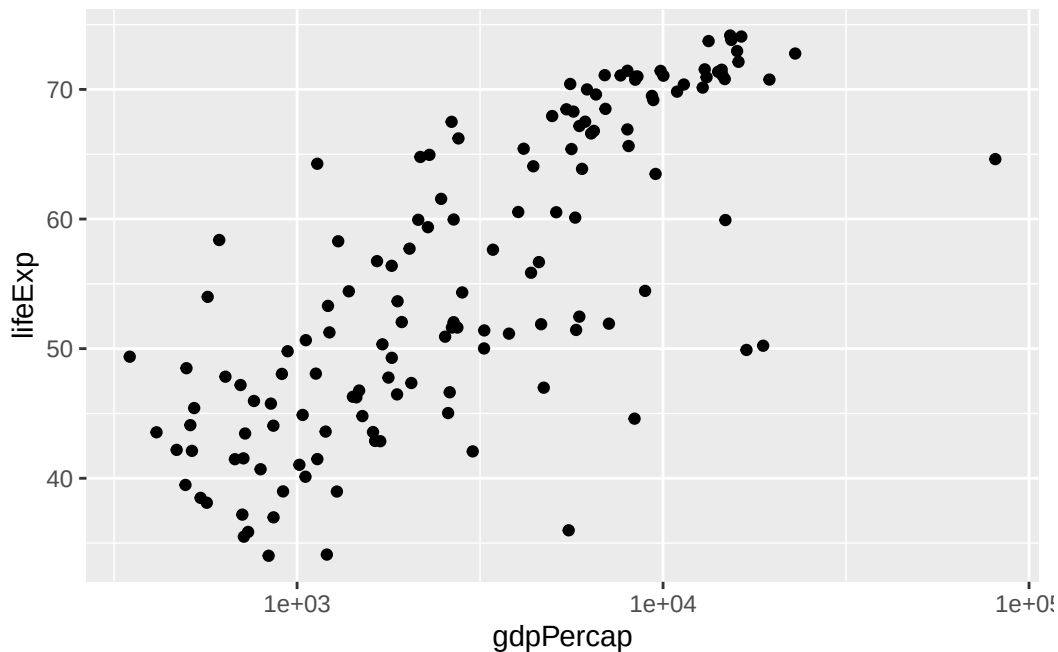
Let's explain much more deeper how to work with Aesthetics using the last example seen.

```
library(tidyverse)
library(gapminder)
```

Aesthetics for defining axis

```
# Generating data to show
gapminder67 <- gapminder[gapminder$year == 1967, ]

# Base plot
ggplot(data = gapminder67) +
  geom_point(mapping = aes(y = lifeExp, x = gdpPercap)) +
  scale_x_log10()
```



In the base plot, we defined inside the aesthetics the x and y axis to be used in the plot. Aesthetics
But we have more arguments that can be used:

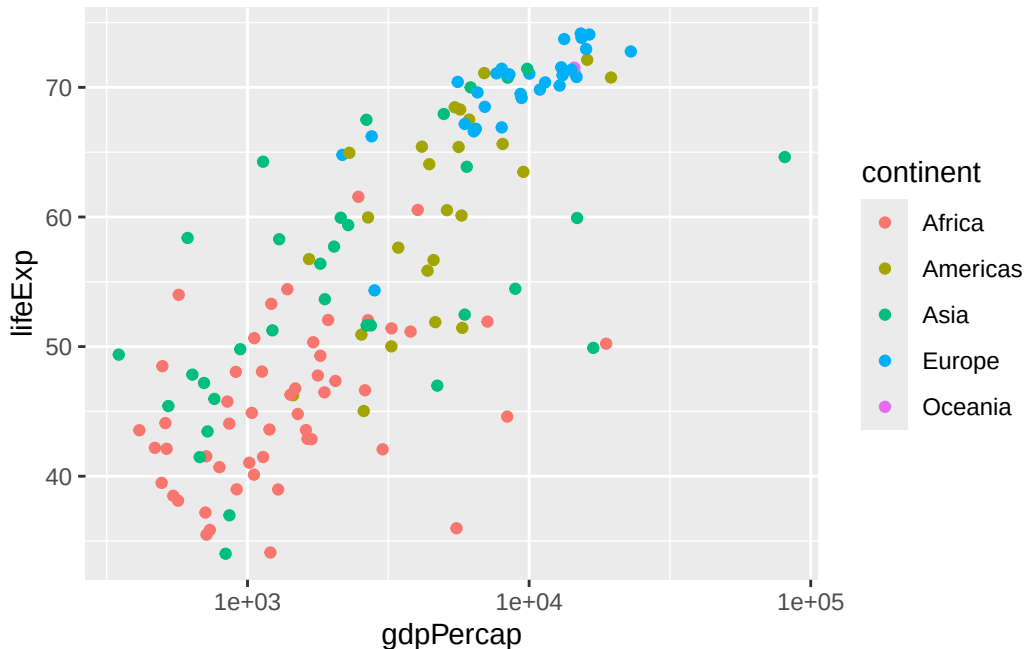
- **shape**
- **color** / **fill** (for borders or for full figure)
- **size**
- **alpha** (transparency)

Depending on the geometry and type of data (continuous or discrete) we are using this parameters work differently and not all of them can be used in all aesthetics. Let's see some examples with our current base plot.

Coloring the figures: color

In some cases we will want that the plot to be colored following the data or categories we have in the data set. In this case we can use color or fill parameter inside the aesthetics to add this information.

```
ggplot(data = gapminder67) +  
  geom_point(mapping = aes(y = lifeExp, x = gdpPerCap, color=continent)) +  
  scale_x_log10()
```



In this case we have added color = continent argument inside the aes function of the geometry, this way the different possible values of continent will be used for coloring the dots.

Exercise: Color with a continuous variable

Try now to do the same now using pop column instead of continent, what happens? Hint: pop is a continuous variable so this may affect how the aesthetic work.

Inside and outside the aesthetics

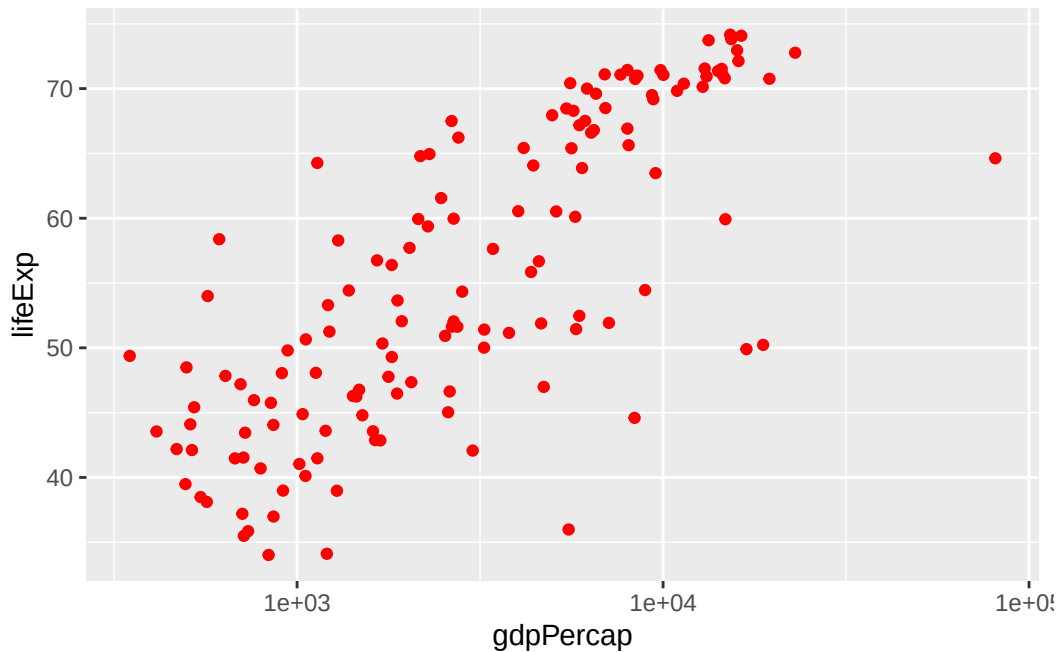
Some parameters can be defined outside of the aesthetics block, this depends if we want to use data from the data frame or if we want to use a constant value.

Parameters color vs fill

- **Inside aesthetics:** We want the figures to be colored following another data value or category.
- **Outside aesthetics:** We want to use an static value for the parameter.

In the example of color we have colored our values following another data, but we can color all dots with the same color if we want to:

```
ggplot(data = gapminder67) +  
  geom_point(mapping = aes(y = lifeExp, x = gdpPerCap), color="red") +  
  scale_x_log10()
```

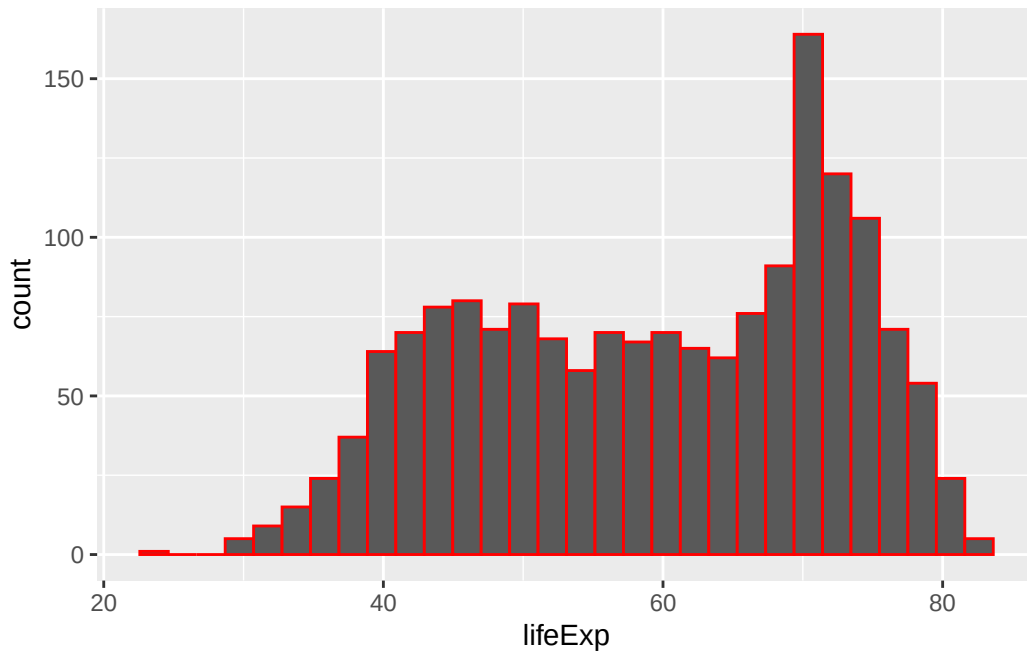


Parameters color vs fill

In some cases we have two possible options when coloring the plots, depending on the geometry both parameters work differently. For example, using the same histogram of the example:

```
ggplot(data = gapminder) +  
  geom_histogram(mapping = aes(x = lifeExp), color = "red")
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.

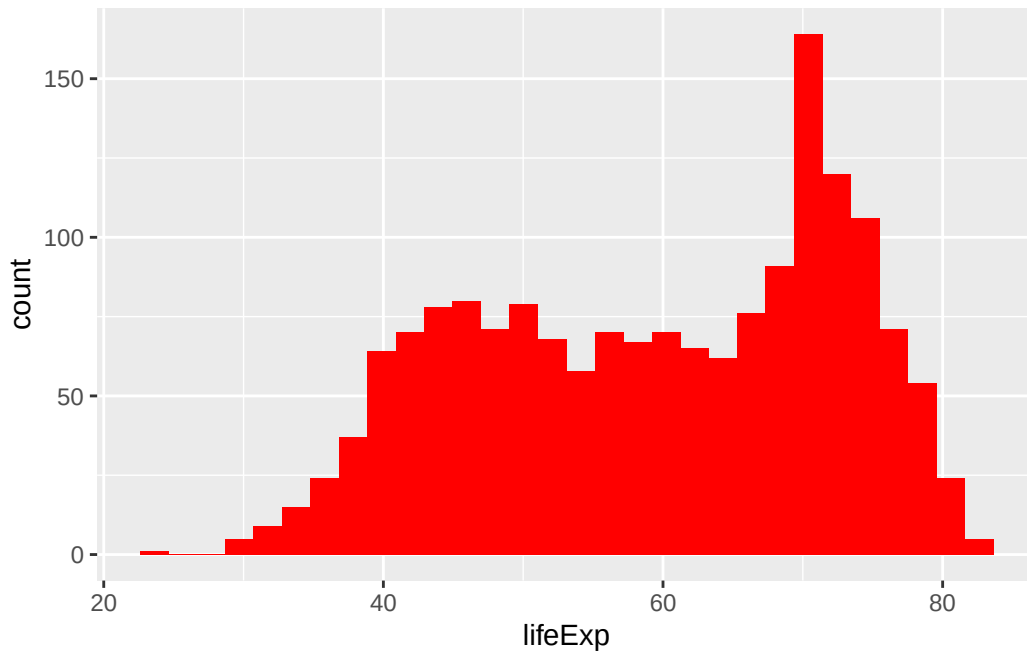


Using color with an histogram will only change the border of the bins or bars. If we want to color the full bin we need to use fill.

```
ggplot(data = gapminder) +  
  geom_histogram(mapping = aes(x = lifeExp), fill = "red")
```

``stat_bin()`` using ``bins = 30``. Pick better value ``binwidth``.

Shaping the figures: shape

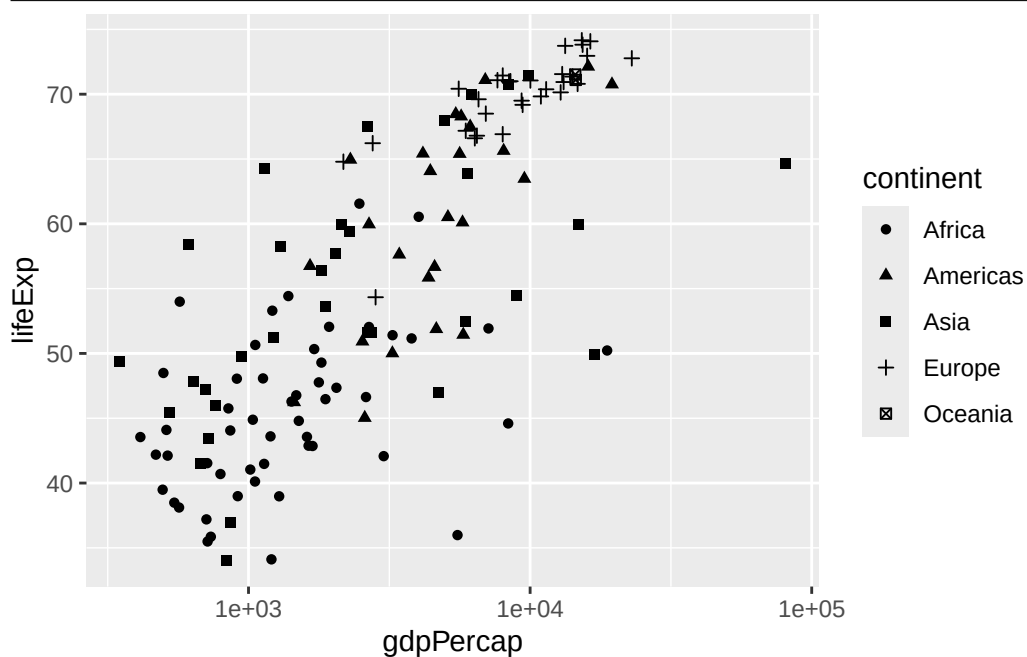


This behavior will change depending on the geometry we are using, for example, with the `geom_point` we can use color for the whole dot not using `fill`. You can know more about which aesthetics can be used reading the documentation of each geometry.

Shaping the figures: shape

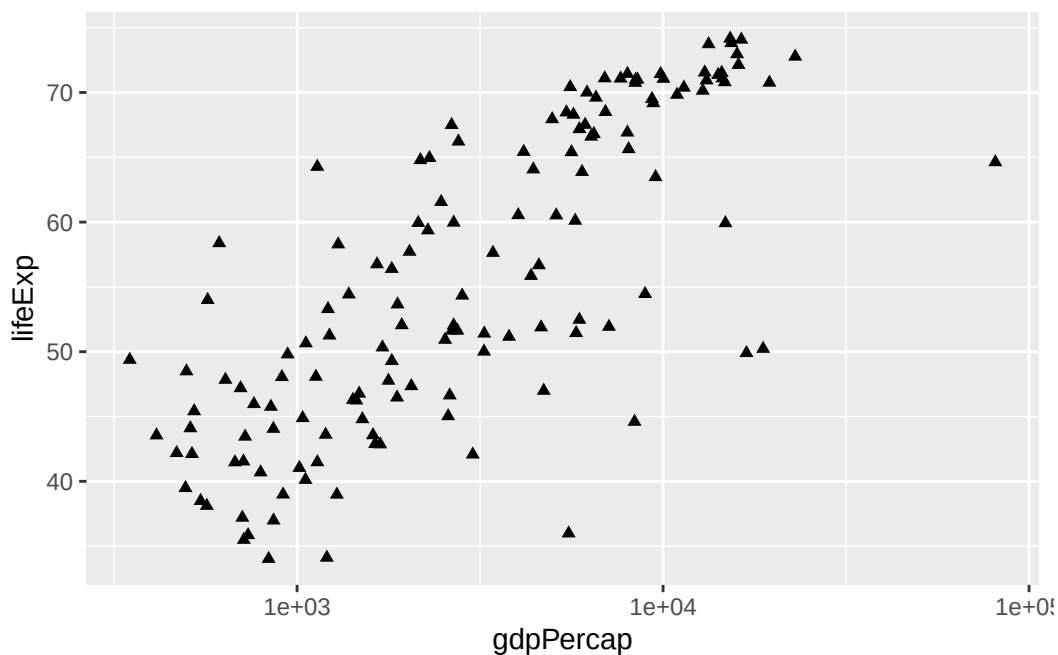
We can change the shape of the certain geometries depending on the data or category. We can do it using `shape`, each group or category will have a different shape.

```
ggplot(data = gapminder67) +  
  geom_point(mapping = aes(y = lifeExp, x = gdpPerCap, shape=continent)) +  
  scale_x_log10()
```



And you can define a single shape for every dot too, using the argument outside of the aesthetics...

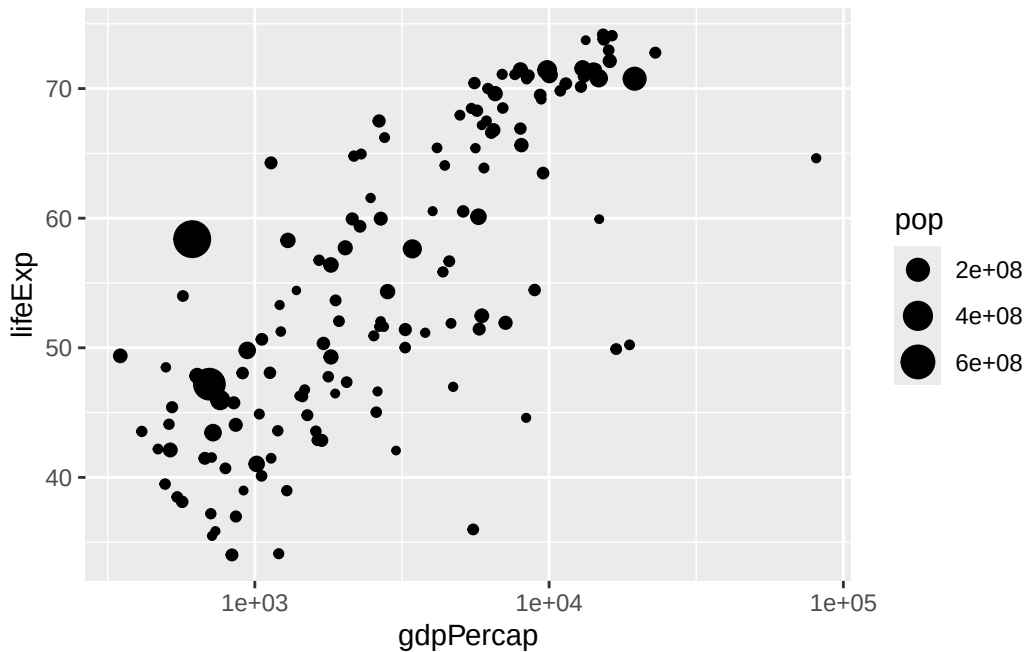
```
ggplot(data = gapminder67) +  
  geom_point(mapping = aes(y = lifeExp, x = gdpPerCap), shape='triangle') +  
  scale_x_log10()
```



Changing the size using size

Another common resource is changing the size of the elements following certain data. This is useful for continuous variables, we can see this example.

```
ggplot(data = gapminder67) +  
  geom_point(mapping = aes(y = lifeExp, x = gdpPercap, size=pop)) +  
  scale_x_log10()
```



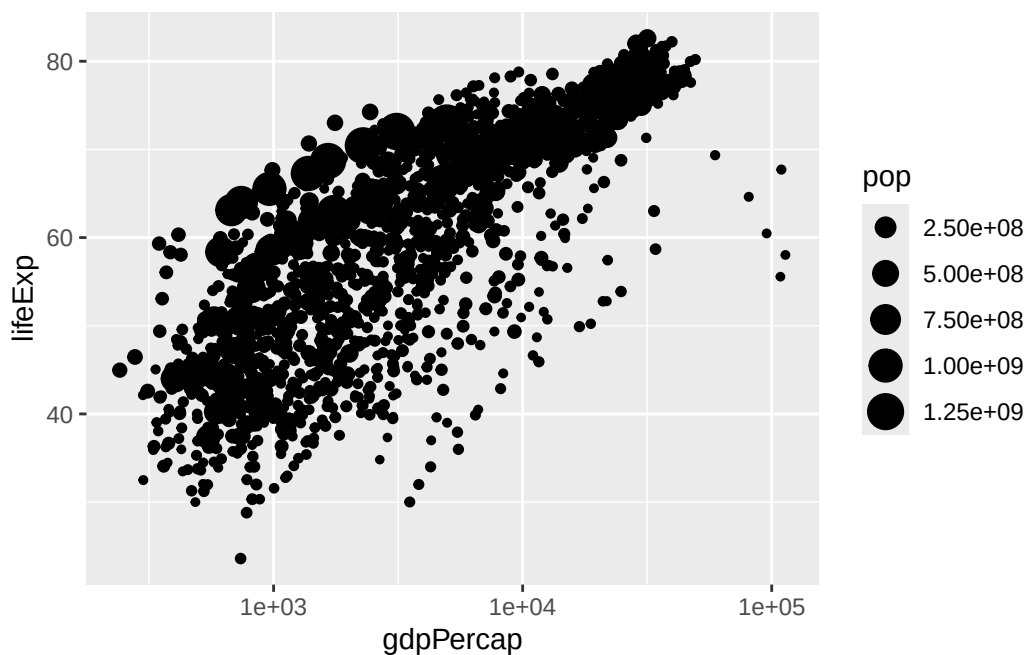
You can define a unique size for all dots using the size argument outside aesthetics and providing a single number.

Adding transparency using alpha

In some cases our visuals will show overlapping elements, difficulty the analysis and vision of the data. We can add some transparency using alpha to make elements more visible when they have another one overlapping it.

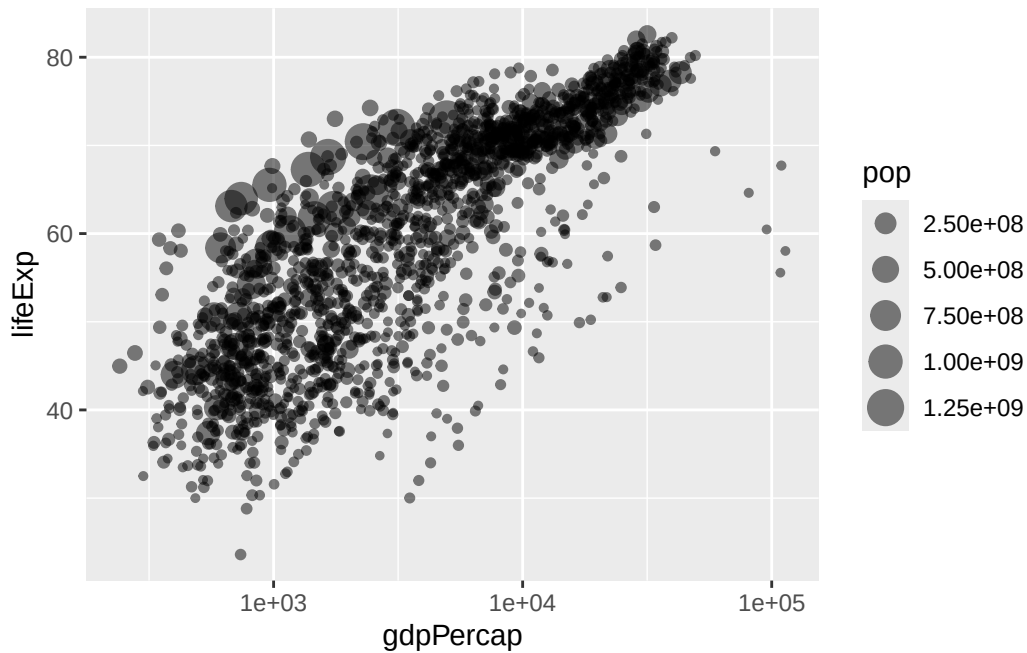
Check this visual, we are using the full gapminder data and some dots are hiding another ones making difficult the analysis.

```
ggplot(data = gapminder) +  
  geom_point(mapping = aes(y = lifeExp, x = gdpPercap, size=pop)) +  
  scale_x_log10()
```



For improving the visualization we can use *alpha*, it takes a value between 1 to 0, 1 no transparency and 0 full transparency.

```
ggplot(data = gapminder) +  
  geom_point(mapping = aes(y = lifeExp, x = gdpPercap, size=pop),  
             alpha = 0.5) +  
  scale_x_log10()
```



Now we can see more clearly all dots including those that are below another one.

To Sum Up

We can represent both continuous and discrete variables using aesthetics, here you can find a summary of the main possible aesthetics you can use to customize the plot and how will be its behavior depending on the variable type:

aesthetic	discrete	continuous
color / fill	different colors	gradient coloring
shape	different shapes	doesn't work with
size	discrete steps	linear mapping

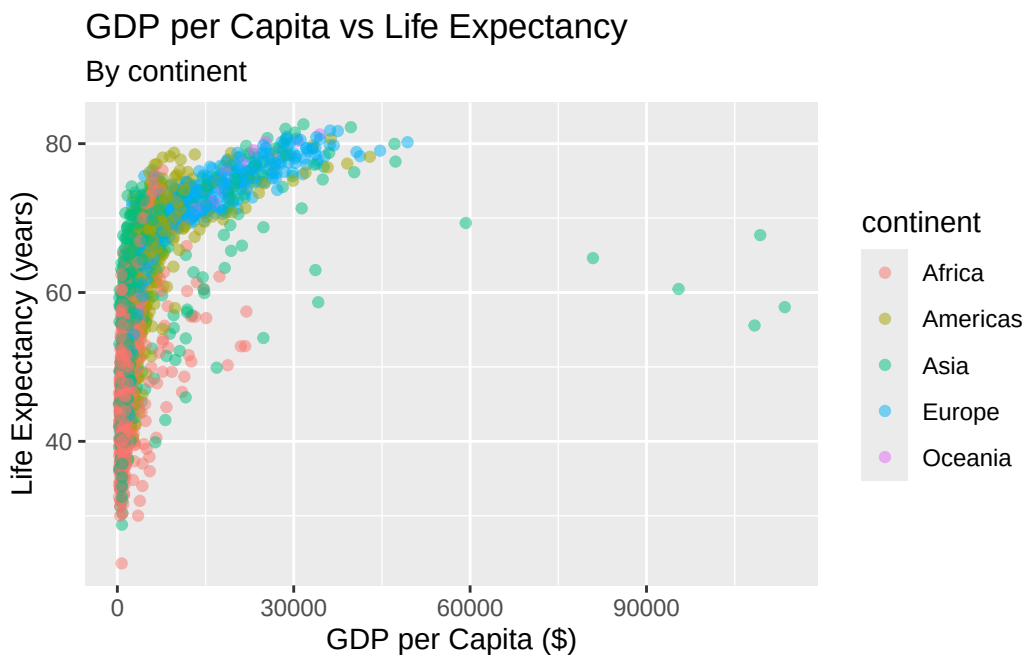
Facets

In some cases we will want to develop different sub-plots into the same plot or show the information separated for better visualization.

Imagine that we are working with the same plot as the last topics covered:

```
library(tidyverse)
library(gapminder)

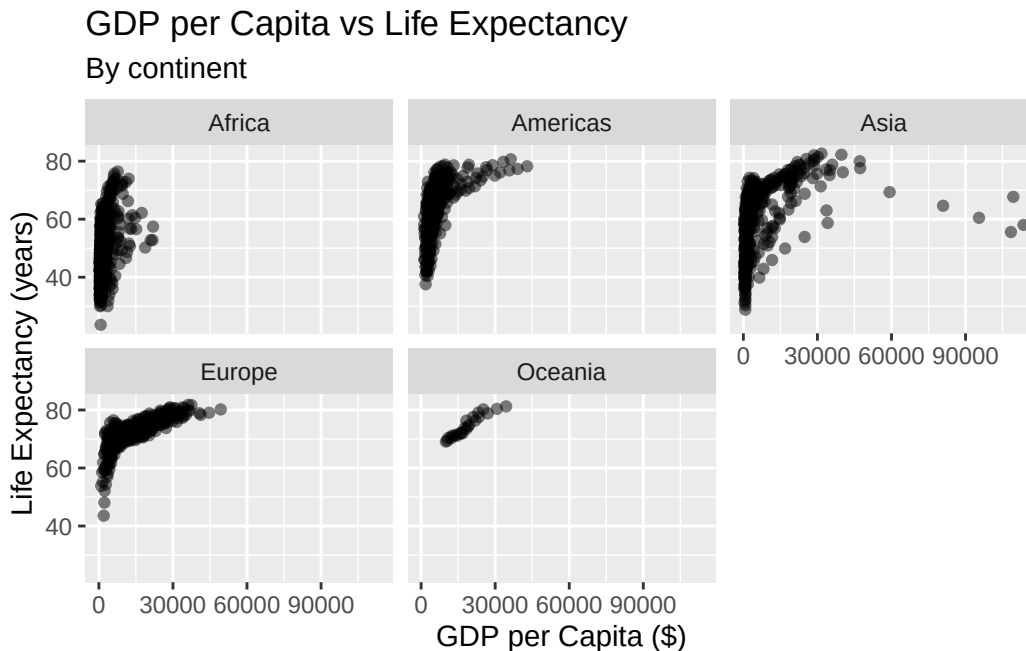
ggplot(data = gapminder, aes(x = gdpPercap, y = lifeExp, color = continent)) +
  geom_point(alpha=0.5) +
  labs(title = "GDP per Capita vs Life Expectancy",
        subtitle = "By continent",
        x = "GDP per Capita ($)", y = "Life Expectancy (years)")
```



But for improving our visualization we want to see it separated for each continent instead of all the information on the same plot.

```
ggplot(data = gapminder, aes(x = gdpPercap, y = lifeExp)) +
  geom_point(alpha=0.5) +
  labs(title = "GDP per Capita vs Life Expectancy",
        subtitle = "By continent",
```

```
x = "GDP per Capita ($)", y = "Life Expectancy (years)" +  
facet_wrap(vars(continent))
```



Using this configuration we can create different plots for each category, continent in this case, and analyze the information much better.

Depending on the type of facet we want to generate we can play with `facet_grid` or `facet_wrap`.

Faceting using Grids: `facet_grid`

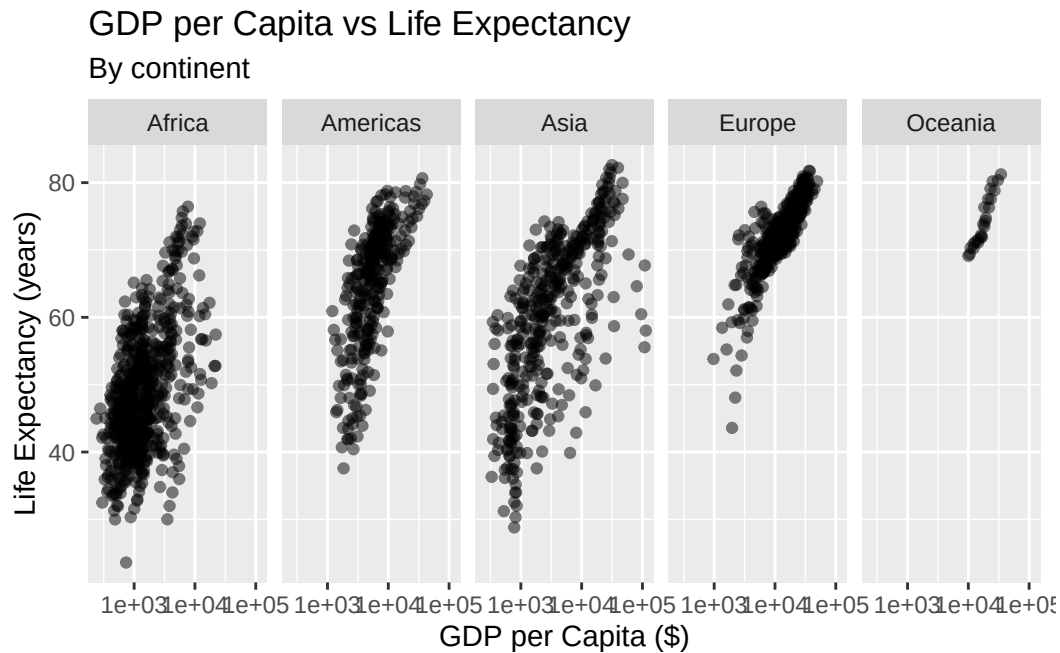
Facet Grid works dividing the ggplot canvas in different areas creating a grid of rows and columns. For using it we need to use the function `facet_grid()` and provide to the grid the facets, the variables to use for generating the subplot.

For defining the facets we need to follow this format `rows ~ columns`, where rows will be the variable we want to use for generating subplots in each row, columns the same but per column. We can define only one of them or both.

For example we want to see the information per continent showing all continents in columns...

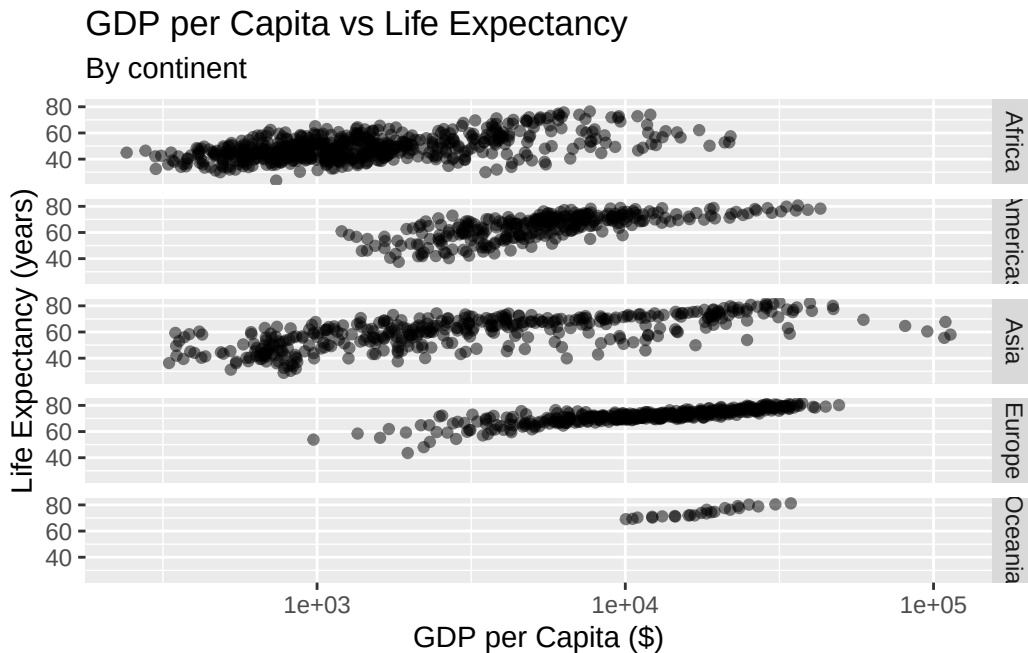
```
ggplot(data = gapminder, aes(x = gdpPercap, y = lifeExp)) +  
  geom_point(alpha=0.5) +  
  labs(title = "GDP per Capita vs Life Expectancy",  
       subtitle = "By continent",
```

```
x = "GDP per Capita ($)", y = "Life Expectancy (years)" +
scale_x_log10()+
facet_grid(.~continent)
```



The other way, we want to see the information per continent showing all continents in rows...

```
ggplot(data = gapminder, aes(x = gdpPercap, y = lifeExp)) +
  geom_point(alpha=0.5) +
  labs(title = "GDP per Capita vs Life Expectancy",
       subtitle = "By continent",
       x = "GDP per Capita ($)", y = "Life Expectancy (years)") +
  scale_x_log10()+
  facet_grid(continent~.)
```



As you can see, when we do not use any of the dimensions we type `.` so R understands that we do not use that dimension.

Exercise

Try now to generate the same plot but using two dimensions, so you see all continents in columns and the year as rows. This way we will have one plot for each year for each continent. Use the new `gapminder_90` data frame generated on the code below.

Faceting using Wraps: `facet_wrap`

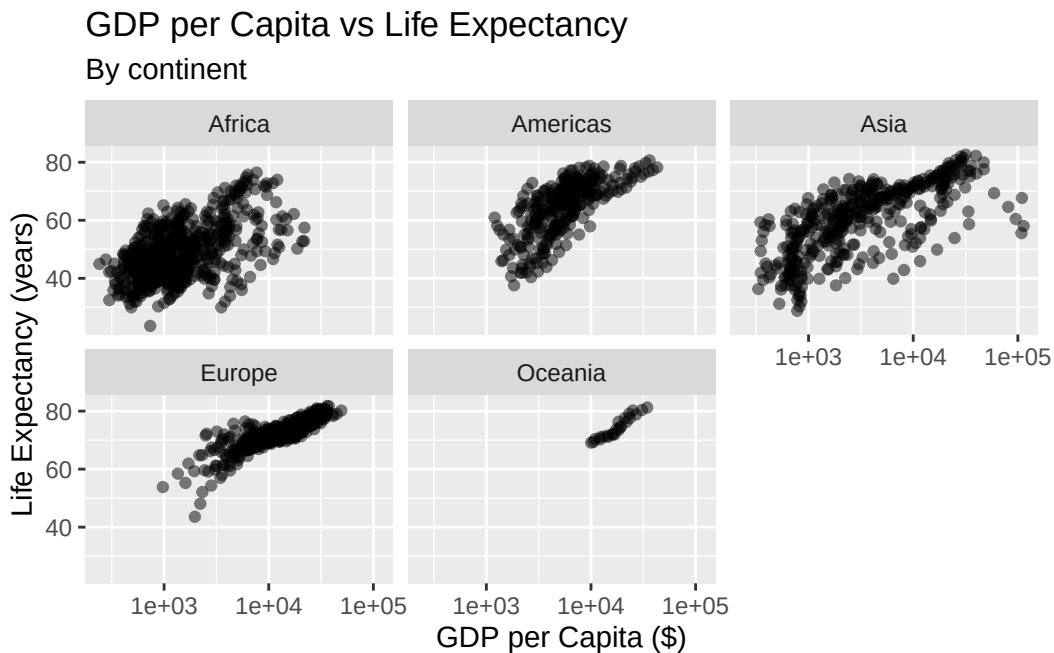
Facet Wrap works generating plots randomly and not following any defined structure like the Grid system. With wrap you can create different subplots and they will be generated one by one.

In this case we do not follow the rows and columns system, instead we use `vars()` function to provide the variable we want to use.

Same plot using continent in columns section.

```
ggplot(data = gapminder, aes(x = gdpPercap, y = lifeExp)) +
  geom_point(alpha=0.5) +
  labs(title = "GDP per Capita vs Life Expectancy",
       subtitle = "By continent",
       x = "GDP per Capita ($)", y = "Life Expectancy (years)") +
```

```
scale_x_log10()+
facet_wrap(vars(continent))
```



Consider that you can use the same rows and columns syntax, but the output won't change as `facet_wrap` doesn't consider any structure.

🔥 Exercise

Try now to generate the same plot but using two dimensions as the last exercise. This way we will have one plot for each year for each continent. Use the same `gapminder_90` dataframe of the last exercise.

Freeing the scales

By default when the plots are being generated you will see that all plots have fixed axis, and share the same scales. This is perfect for improving comparability and read the data into the same scaling.

In some cases we will want that each plot has different scales and is adjusted to the data being displayed. This is only fully possible using `facet_wrap`. As it doesn't follow any grid structure and makes possible separating the axis, in `facet_grid` we can only do it on the not shared axis.

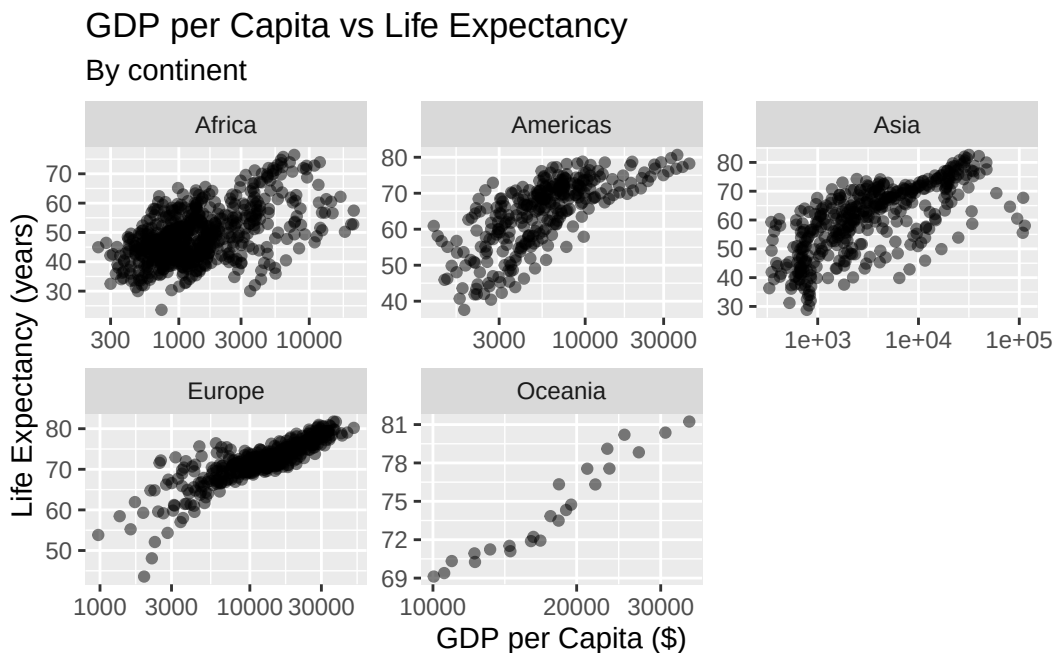
For doing that there is an argument called `scales` inside the facet function. This can be:

- “fixed” by default to fix all scales and share them.

- “free” to free both x and y so they are independent for each plot.
- “free_x” or “free_y” to free only x or y, and share the other axis.

So imagine we want to free all axis of the plot, it will look like this:

```
ggplot(data = gapminder, aes(x = gdpPercap, y = lifeExp)) +
  geom_point(alpha=0.5) +
  labs(title = "GDP per Capita vs Life Expectancy",
       subtitle = "By continent",
       x = "GDP per Capita ($)", y = "Life Expectancy (years)") +
  scale_x_log10()+
  facet_wrap(vars(continent), scales = "free")
```



Exercise

Repeat the last plot but now:

1. Use `facet_wrap` and free y.
2. Use `facet_wrap` and free x.
3. Use `facet_grid` showing continent on columns and free x.
4. Use `facet_grid` showing continent on columns and free y, it is really free?

To Sum Up

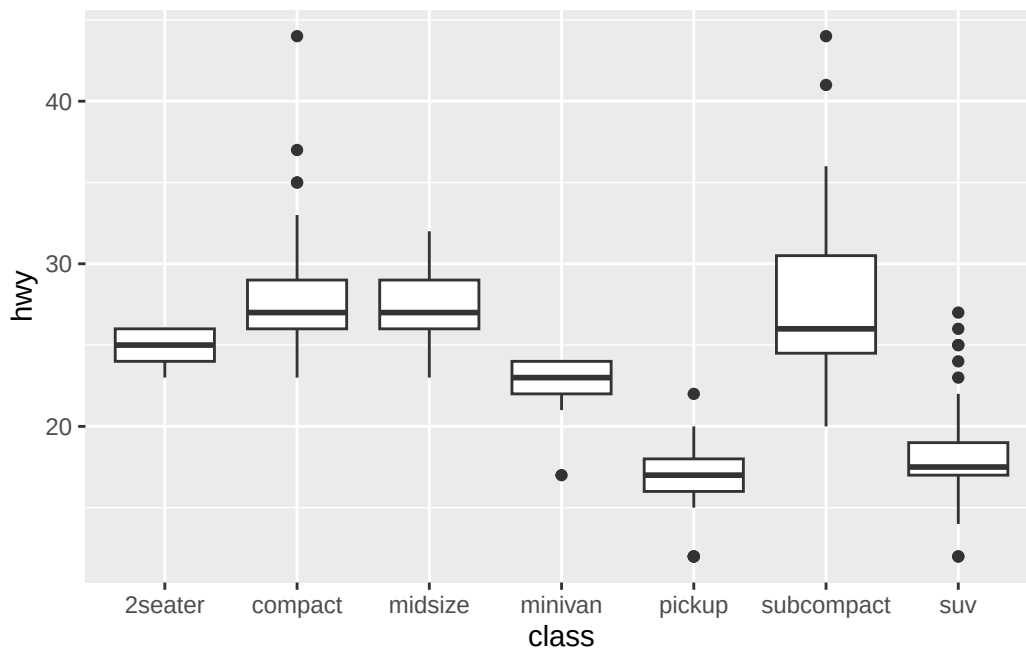
- **facet_grid()**
 - 2 dimensions: **rows ~ columns**
 - use **.** to avoid using one of them
- **facet_wrap()**
 - 1 dimension
 - use **scales = ("free_x", "free_y", "free")** to fix the scales of the different subplots

Other Features

Coordinates

By default ggplot uses Cartesian coordinates (x,y), but you can modify them easily using `coord_XXXX()` functions.

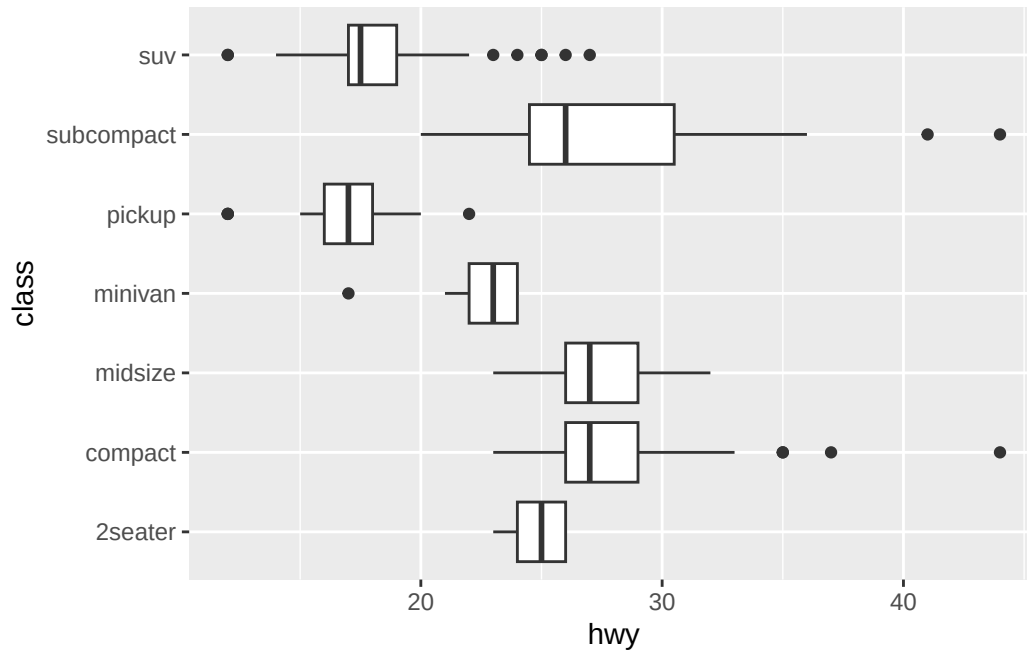
```
ggplot(data = mpg,  
       mapping = aes(x = class, y = hwy)) +  
  geom_boxplot()
```



```

1 ggplot(data = mpg,
2       mapping = aes(x = class, y=hwy)) +
3   geom_boxplot() +
4   coord_flip()

```



Maps

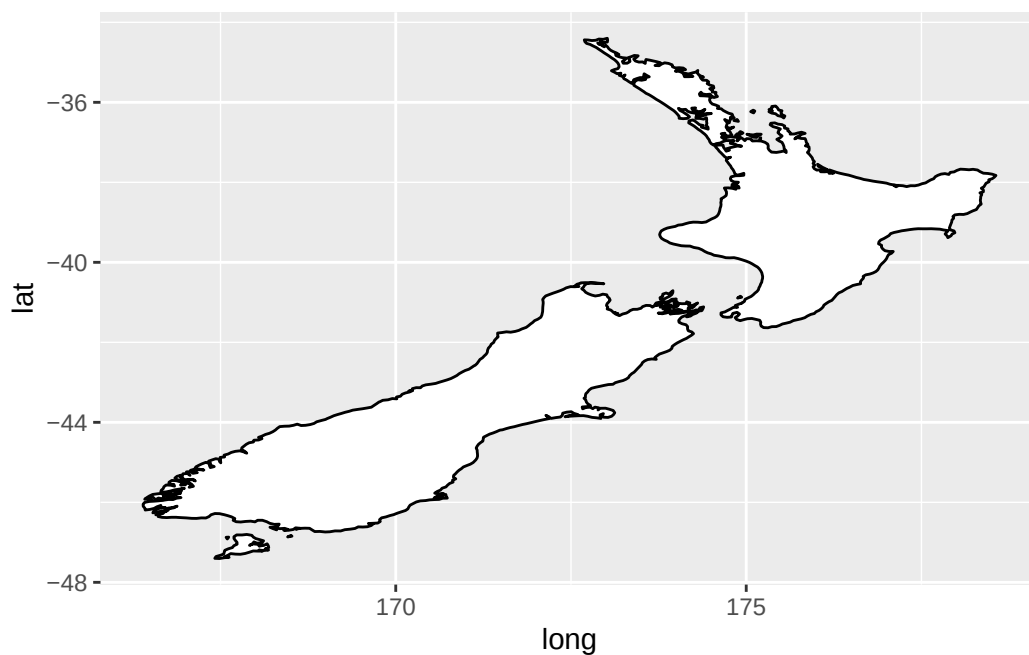
You can use maps for your plots, and with `coord_quickmap()` it will compute the accurate ratio for the maps.

```

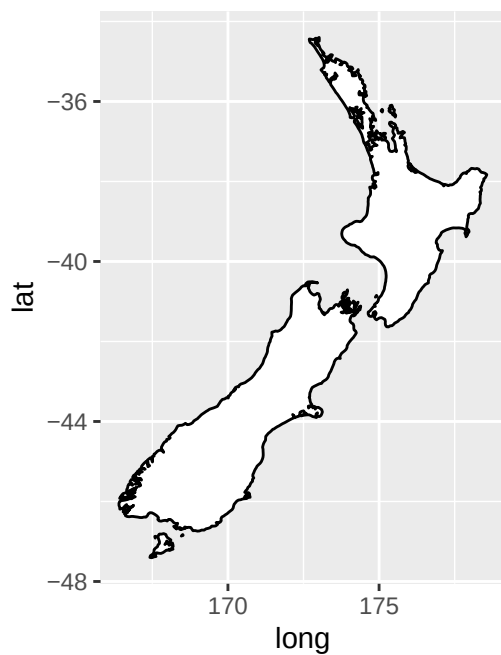
library(maps)
nz <- map_data("nz")

ggplot(nz, aes(long, lat, group = group)) +
  geom_polygon(fill = "white",
              colour = "black")

```



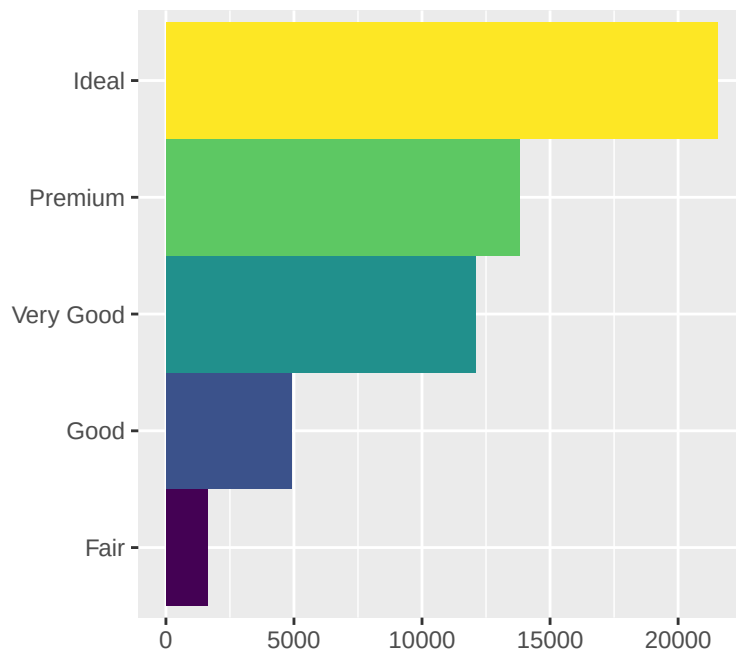
```
1 ggplot(nz, aes(long, lat, group = group)) +  
2   geom_polygon(fill = "white",  
3               colour = "black") +  
4   coord_quickmap()
```



Polar Plots

We can change how to represent some data, using another visualization style. Imagine we have the following plot...

```
ggplot(diamonds) +  
  geom_bar(  
    aes(x = cut, fill=cut),  
    show.legend = F,  
    width = 1  
  ) +  
  theme(aspect.ratio = 1) +  
  labs(x=NULL, y=NULL) +  
  coord_flip()
```



We can change it to much more interesting style...

```
1 ggplot(diamonds) +  
2   geom_bar(  
3     aes(x = cut, fill=cut),  
4     show.legend = F,  
5     width = 1  
6   ) +  
7   theme(aspect.ratio = 1) +
```

Polar Plots

```
8 labs(x=NULL, y=NULL) +  
9 coord_polar()
```

